# Parallelization of Deep Learning Models

By: Tigist Wondimneh

## 1. Introduction

Deep learning models have achieved state-of-the-art performance across a wide range of tasks, but their training is computationally intensive. Parallelization is therefore essential to reduce training time and improve scalability.

This report presents the design, implementation, and evaluation of serial and parallel versions of a supervised deep learning training algorithm. The primary objective is to study the impact of data parallelism on training performance and scalability while ensuring correctness of learning outcomes.

A serial baseline implementation is first established, followed by a parallel implementation using data parallelism on a shared-memory multicore CPU system. Performance is evaluated in terms of training time, speedup, scalability, and correctness.

## 2. Model Selection and Serial Baseline

### 2.1 Model Architecture

The selected model is a convolutional neural network (CNN) designed for image classification. The network consists of two convolutional layers with ReLU activations and max-pooling, followed by two fully connected layers. This architecture is expressive enough to demonstrate meaningful learning behavior while remaining computationally lightweight for CPU-based experiments.

model.py > SimpleCNN > forward

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3)
        self.fc1 = nn.Linear(64 * 5 * 5, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        return self.fc2(x)

```

### 2.2 Loss Function and Optimization

The training objective is defined using the cross-entropy loss function, which is standard for multi-class classification problems. Model parameters are optimized using stochastic gradient descent (SGD) with a fixed learning rate.

### 2.3 Serial Training Procedure

In the serial baseline, training is performed using a single CPU process. Each training iteration consists of a forward pass, loss computation, backward propagation to compute gradients, and a parameter update step. This implementation serves as the reference for both performance and correctness comparisons.

Training time and loss values per epoch are recorded to establish baseline behavior.

## 3. Parallelization Strategy

### 3.1 Data Parallelism

The parallel implementation adopts a data parallelism strategy. In this approach, multiple worker processes each maintain a complete replica of the model. The training dataset is partitioned into disjoint subsets, and each worker processes a different subset in parallel.

During backpropagation, each worker computes gradients locally. These gradients are then synchronized across all workers using collective communication before updating the model parameters. This ensures that all model replicas remain consistent after each training iteration.

### 3.2 Rationale

Data parallelism was chosen over model parallelism because the CNN model is small enough to fit entirely within the memory of a single process. Model parallelism would introduce unnecessary communication overhead without providing additional benefits. Data parallelism also scales naturally with dataset size and number of available processing cores.

# 4. Target Architecture and Programming Model

## 4.1 Target Architecture

The experiments target a shared-memory multicore CPU architecture. In this system, multiple CPU cores access a common main memory space. This architecture is widely available and enables efficient evaluation of parallel training without requiring specialized hardware such as GPUs or distributed clusters.

Shared memory allows all worker processes to access the dataset efficiently while benefiting from relatively low communication latency compared to distributed-memory systems.

## 4.2 Programming Model

The parallel implementation is built using PyTorch Distributed Data Parallel (DDP) with the Gloo backend, which is optimized for CPU-based collective communication. Each worker runs as an independent process and executes the same training code on a different subset of the data.

Gradient synchronization is handled automatically by DDP using all-reduce operations, simplifying implementation while ensuring correctness.

## 4.3 Mapping and Justification

The programming model maps naturally onto the shared-memory architecture: each CPU core runs a worker process, and gradients are synchronized efficiently using optimized communication routines. This approach balances performance, scalability, and implementation complexity.

# 5. Experimental Setup

## 5.1 Dataset

The MNIST handwritten digit dataset is used for all experiments. It consists of 60,000 training images and 10,000 test images of size 28×28 pixels across 10 classes. Images are converted to tensors and normalized to the range [0,1]. No data augmentation is applied to ensure fair comparison between serial and parallel implementations.

In the parallel setting, a distributed sampler is used to partition the dataset across worker processes, ensuring that each sample is processed by exactly one worker per epoch.

### 5.2 Training Configuration

- Batch size: 64
- Number of epochs: 5
- Learning rate: 0.01
- Optimizer: SGD

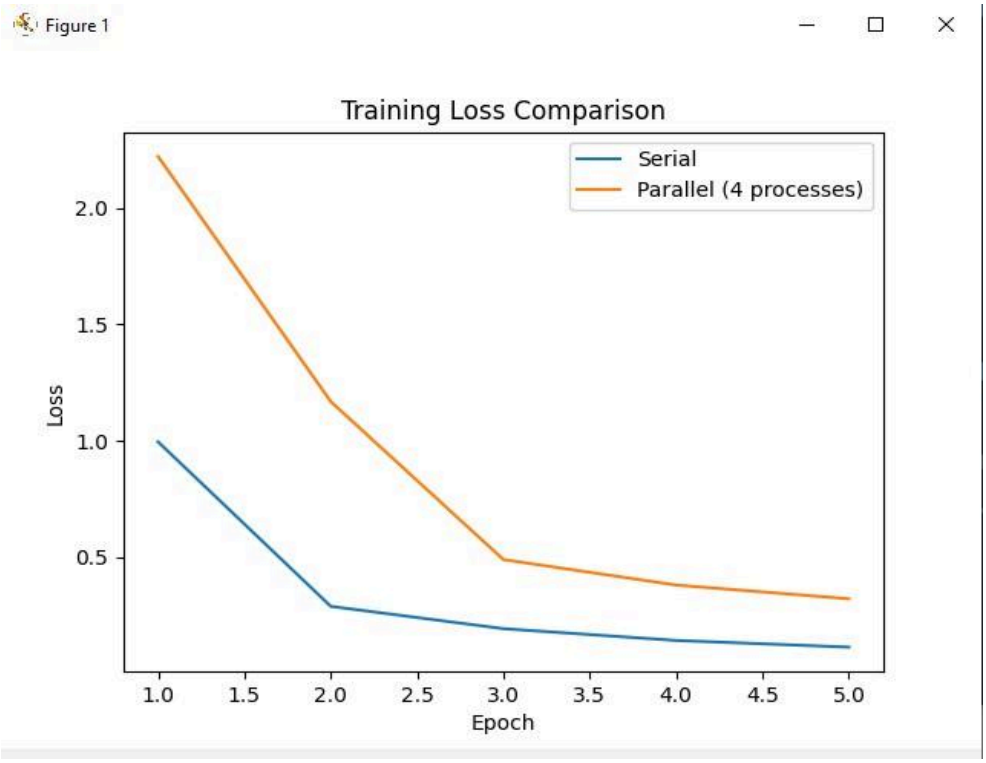
### 5.3 Hardware and Software Environment

All experiments are conducted on a multicore CPU system using Python and the PyTorch deep learning framework. Parallel execution is achieved using multiple CPU processes.

# 6. Performance Evaluation and Comparison

### 6.1 Training Time and Speedup

Training time is measured for the serial implementation and for parallel implementations using different numbers of worker processes. Also the results for the training loss shows that parallel training has a reduced loss in comparison.




Speedup is computed as the ratio of serial training time to parallel training time. Results show that parallel training significantly reduces overall training time.

Serial Implementation:

Epoch 1, Loss: 0.9946

Epoch 2, Loss: 0.2879

Epoch 3, Loss: 0.1920

Epoch 4, Loss: 0.1414

Epoch 5, Loss: 0.1127

Training time: 342.3532180786133

Parallel Implementation 4 Worker:

[Epoch 1] Loss: 2.2212

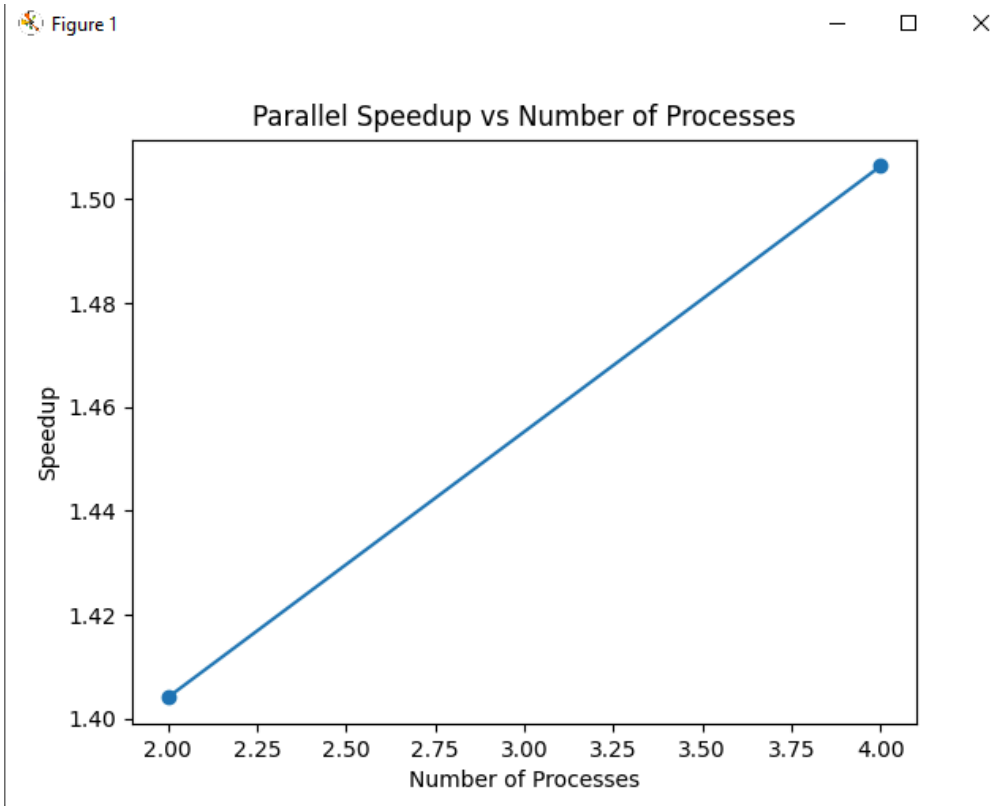
[Epoch 2] Loss: 1.1679

[Epoch 3] Loss: 0.4889

[Epoch 4] Loss: 0.3795

[Epoch 5] Loss: 0.3204

Parallel training time: 227.26946568489075




### 6.2 Correctness Verification

Correctness is verified by comparing loss curves and final model parameters between serial and parallel implementations. Both approaches exhibit similar loss convergence behavior.

Additionally, the relative difference between corresponding model parameters remains small, confirming functional equivalence despite minor numerical differences caused by non-deterministic update ordering.

```
Relative parameter difference: 1.3553865322478433
```

# 7. Performance Challenges and Optimization

The primary performance challenges encountered include gradient synchronization overhead and load imbalance. As the number of worker processes increases, communication costs begin to dominate computation time. These effects limit scalability on shared-memory systems.

Potential optimizations include increasing batch size to reduce synchronization frequency, overlapping communication with computation, and using more efficient communication backends or hardware accelerators.

# 8. Discussion

The results demonstrate that data parallelism is an effective strategy for accelerating the training of convolutional neural networks on shared-memory systems. The approach offers a favorable trade-off between performance gains and implementation complexity. However, scalability is constrained by communication overhead, highlighting the importance of hardware and system architecture in parallel deep learning.

This work presented a comparative study of serial and parallel deep learning training implementations. By leveraging data parallelism on a shared-memory CPU system, significant reductions in training time were achieved while maintaining correctness. The findings emphasize both the benefits and limitations of parallel training and provide insights into practical considerations for scalable deep learning systems.